

代理对接文档

让你自己的充值站直接完成兑换：买家在你的系统里粘贴卡密和账号信息，你的服务器调用本接口，买家不用再跳到我们的网站。

版本 v1 · 2026-10-05 · 接口地址 `https://recharge.chongplus.store/api/open/v1`

1. 流程总览

一次兑换是四步，每一步都是你的服务器向我们发一个 HTTPS 请求：

1 查卡密（可选）

买家粘贴卡密后，调用 `POST /cdk/query`，告诉买家这是什么套餐、能不能用。不消耗卡密。

2 核对账号

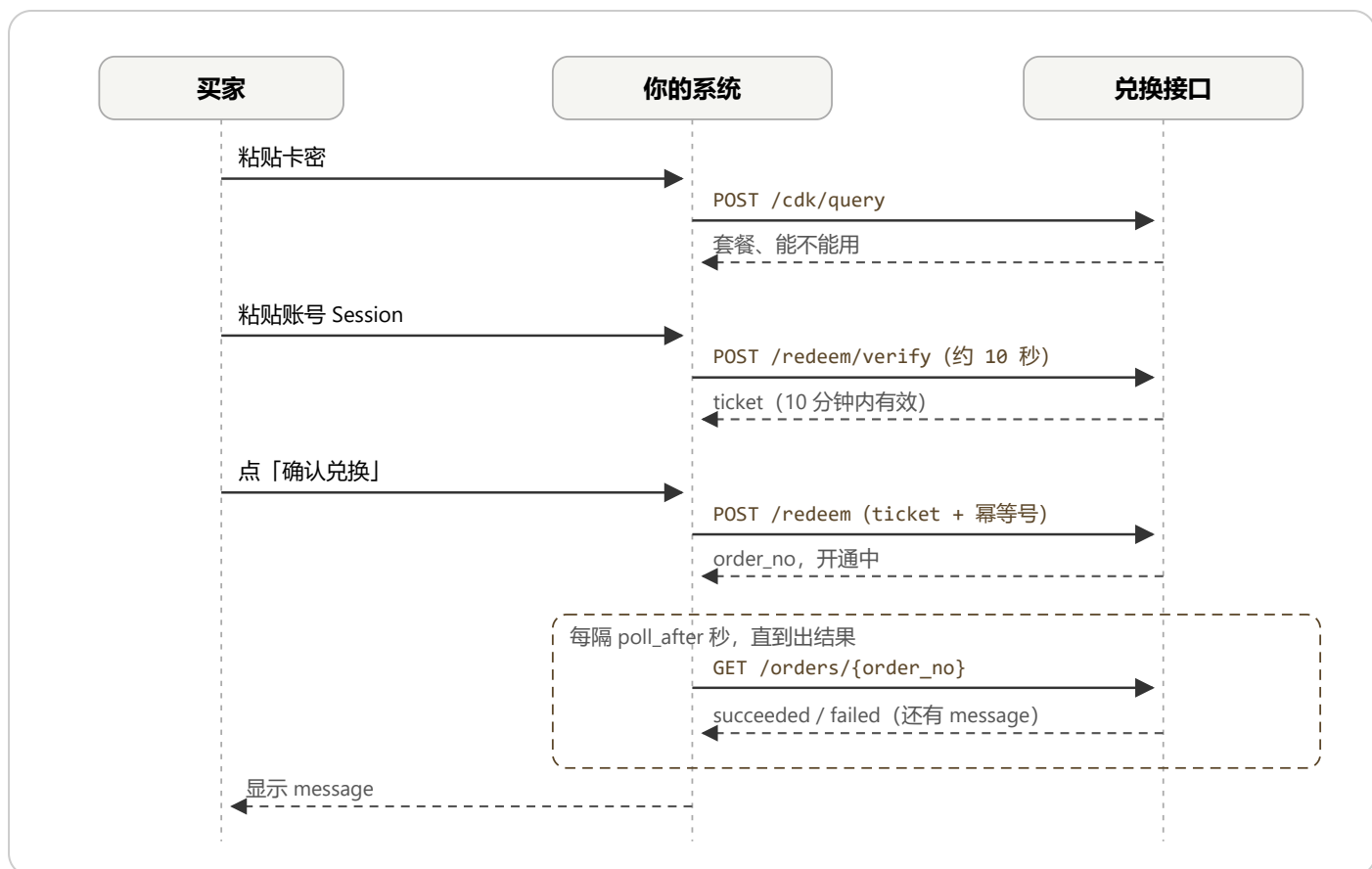
买家粘贴自己的 ChatGPT 账号信息 (Session) 后，调用 `POST /redeem/verify`。约 10 秒，通过后拿到一张 `ticket`（核对票据，10 分钟内有效）。

3 确认兑换

买家点「确认兑换」，调用 `POST /redeem`，带上 `ticket` 和你生成的 `idempotency_key`（幂等号），拿到订单号 `order_no`。

4 查订单直到出结果

按返回的 `poll_after` 秒轮询 `GET /orders/{order_no}`，直到 `status` 变成 `succeeded`（已开通）或 `failed`（失败）。通常 1-3 分钟。



一次交换的时序。实线是请求，虚线是返回。

最重要的一条

网络超时、没拿到响应的时候，**用同一个 idempotency_key 原样再发一次**（或用 `POST /orders/lookup` 找回订单），绝不会重复兑换；**不要**换新的幂等号、也不要重新核对再提交。细则见 [POST /redeem](#)。

2. 开始之前

我们会给你三样东西：

项目	说明
接口地址	<code>https://recharge.chongplus.store/api/open/v1</code> （下文简写 <code>{BASE}</code> ）
API 密钥	<code>cpa_</code> 开头、共 47 个字符。只在创建时显示一次，我们这边只存指纹，丢了只能停用后重发。
来源 IP 登记	把你服务器的 出口 IP （或 IP 段）告诉我们。登记后，这把密钥只有从这些 IP 发来的请求才有效。不知道出口 IP 的话：用 <code>GET /ping</code> 可以看到我们看到的你的来源 IP；如果密钥已经限了 IP，被拒绝的回包（ <code>ip_not_allowed</code> ）里的 <code>error.details.ip</code> 也是我们看到的你的来源 IP。

另附一份 OpenAPI 3.0 格式的接口描述文件 `openapi.json`：可以直接导入 Apifox、Postman 这类工具，自动生成全部接口、请求示例和数据结构；里面的字段、取值、错误码和本文档完全一致。

安全规则（请务必遵守）

- 密钥只放在你的**服务器**上（环境变量 / 密钥管理）。不要写进网页、手机 App 或任何前端代码，也不要贴到聊天群、工单里。
- 接口只接受**服务器对服务器**的调用。浏览器里直接调用会被拒绝（我们不放行任何跨域请求）。
- 只用 `https://` 调用。用 `http://` 的话密钥在路上是明文，等于已经泄露，请马上联系我们停用、换一把新的。
- 买家的 Session 等同于账号登录凭证：只在请求里传这一次，**不要保存、不要写进日志**。
- 怀疑密钥泄露：马上联系我们停用，停用立即生效，我们会发新的。
- 用错误的密钥、或从没登记的 IP 反复请求，我们会临时封锁该来源 IP 10 分钟。

联调步骤

1. 调 `GET /ping`：确认密钥有效，返回的 `ip` 就是你服务器的出口 IP。
2. 向我们要一张测试卡密，用 `POST /cdk/query` 查一次，看到套餐。
3. 准备一个**没有生效中套餐**的测试账号，走一遍 核对账号 → 确认兑换 → 查订单，直到看到 `succeeded`。没有沙箱环境：测试兑换会真实开通、真实用掉这张测试卡密。
4. 把常见的错误走一遍：卡密写错（`cdk_invalid`）、Session 只复制了一半（`session_format`）、同一个幂等号连发两次（拿到同一张订单）、已有套餐的账号（`eligible: false`）。
5. 对照 [上线前自检](#) 逐条确认。

3. 通用约定

请求

- HTTPS, JSON (UTF-8)。POST 请求带 `Content-Type: application/json`。请求体不超过 128 KB（核对账号一步最大，其他接口的请求体都很小）。
- 认证：每个请求都带 `Authorization: Bearer <你的密钥>`。
- 每个请求带一个 `User-Agent`，写你自己系统的名字（比如 `my-shop/1.0`）。有些语言或库自带的默认值会被网关直接拒绝（比如 Python 的 `urllib` 和 `pycurl`、Java 8 及更早的版本、Perl 的 `LWP`）。被拒时是 HTTP 403、回包只有一行纯文本 `error code: 1010`（不是 JSON），请求没有到达接口。
- 请求超时请设 **60 秒以上**（核对账号一步要 10 秒左右）。

响应

成功 (HTTP 200)：

```
{
  "ok": true,
```

```
"data": { ... },
"trace_id": "9F2A41C07B"
}
```

失败:

```
{
  "ok": false,
  "error": {
    "code": "cdk_used",
    "message": "卡密已兑换或已使用",
    "retryable": false,
    "cdk_state": "used",
    "details": { "order_no": "0D3F9A1C07B2D4E5" }
  },
  "trace_id": "9F2A41C07B"
}
```

字段	说明
<code>error.code</code>	错误代号，稳定不变，用它写判断。全部代号见 错误码总表 。
<code>error.message</code>	给终端买家看的中文句子，可以直接显示；想自己写文案就按 <code>code</code> 判断。
<code>error.retryable</code>	同样的操作稍后能不能再做一次。 怎么重来 以各接口的说明为准（有的要重新核对账号）。
<code>error.retry_after</code>	建议等多少秒再试；同时放在 <code>Retry-After</code> 响应头里。只在需要等的错误里出现。
<code>error.cdk_state</code>	这一步之后卡密是什么状态： <code>available</code> 还没消耗 / <code>held</code> 已锁定 / <code>used</code> 已使用。不一定出现。
<code>error.details</code>	可能带两种信息，都不一定出现： <code>order_no</code> ——卡密已被用或正在处理、而且正是你这把密钥提交的订单时，告诉你订单号，用 <code>GET /orders/{order_no}</code> 去查； <code>ip</code> ——来源 IP 被拒绝（ <code>ip_not_allowed</code> ）时，本站看到的你的来源 IP。
<code>trace_id</code>	每个响应都有。联系我们排查问题时请带上它。

HTTP 状态码

状态码	含义

200	成功（ <code>ok: true</code> ）。
400	请求格式不对（ <code>bad_request</code> ），或 Session 格式不对（ <code>session_format</code> ）。
401 / 403	密钥无效或已停用（ <code>invalid_key</code> ）/ 来源 IP 不在登记的名单里（ <code>ip_not_allowed</code> ）。
404	接口、核对票据或订单不存在。
409	当前状态不允许这个操作：卡密已用、核对已过期、票据已用过、账号要等一等.....
422	请求没问题但内容过不了：卡密无效、Session 失效。
429	请求太频繁 / 同时处理的订单太多 / 系统繁忙。按 <code>Retry-After</code> 等。
500 / 503	我们这边暂时处理不了，稍后重试。

限额

- 每把密钥有「每分钟请求数」和「同时处理的订单数」两个上限（默认 60 次/分钟、5 单；实际数值以 `GET /ping` 返回的 `limits` 为准，需要调整联系我们）。超了返回 429（`rate_limited` / `too_many_inflight`）。
- 所有代理共用一部分处理能力，每把密钥最多占其中的一部分。高峰时、或者你这把密钥发得太密时，可能临时返回 429 `busy`：按 `Retry-After` 稍后重试即可，不是你的密钥有问题。

时间格式

所有时间都是 RFC 3339 的 UTC 时间，例如 `2026-10-05T09:00:00Z`。

4. 接口

下面的例子里 `{BASE}` 是接口地址，`$KEY` 是你的密钥，卡密用 `XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX` 代替。

GET /ping

`GET` /ping

检查密钥是否有效，并告诉你：这把密钥叫什么、**我们看到的你的来源 IP**（登记 IP 名单时以它为准）、限额。接入联调的第一步。

```
curl {BASE}/ping -H "Authorization: Bearer $KEY"
```

```
{
  "ok": true,
  "data": {
    "name": "代理甲",
    "ip": "203.0.113.10",
    "server_time": "2026-10-05T09:00:00Z",
    "limits": { "rate_per_min": 60, "max_inflight": 5 }
  },
  "trace_id": "9F2A41C07B"
}
```

POST /cdk/query

POST /cdk/query

查一张卡密：是什么套餐、现在能不能用。**只是查询**：不消耗卡密，也不需要买家的账号信息。同一张卡密 60 秒内的重复查询可能直接返回缓存结果。你这把密钥自己已经提交过这张卡密的订单时，直接告诉你：订单还在处理是 `cdk_not_usable`，已经兑换成功是 `cdk_used`，都带 `error.details.order_no`。

请求字段	说明
<code>cdk</code>	卡密。大小写、首尾空格不敏感。

```
curl -X POST {BASE}/cdk/query -H "Authorization: Bearer $KEY" \
-H "Content-Type: application/json" \
-d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX"}'
```

```
{
  "ok": true,
  "data": { "plan": "pro-5x", "plan_name": "Pro 5x", "cdk_state": "available" },
  "trace_id": "9F2A41C07B"
}
```

`plan` 一共有这几种取值：

plan	plan_name
<code>go</code>	Go
<code>plus</code>	Plus
<code>pro-5x</code>	Pro 5x

pro-20x	Pro 20x
pro-max	Pro Max
other	ChatGPT 套餐（以后新增的套餐会先用这个，不会让你的程序出错）

可能的错误

code	什么时候	怎么办
cdk_invalid	卡密格式不对或不存在	请买家核对卡密
cdk_used	卡密已兑换	如果 error.details.order_no 有值，那是你这把密钥提交的订单，去查它
cdk_not_usable	暂时不能用（已停用、冻结或正在处理）	如果 error.details.order_no 有值，是你这把密钥提交的订单还在处理，去查它；否则请买家联系你，需要时联系我们
busy / service_unavailable / try_later	系统繁忙或暂时不可用	稍后重试

POST /redeem/verify

POST

/redeem/verify

核对买家的账号：这个账号能不能兑换这张卡密。通过就返回一张 ticket 。**这一步要 10 秒左右**，请把超时设长一点。它不会消耗卡密。

请求字段	说明
cdk	卡密。
session	买家的账号信息： 整页 Session 。可以是字符串，也可以直接是 JSON 对象。

怎么让买家拿到 Session

让买家在已登录 ChatGPT 的浏览器里打开 `https://chatgpt.com/api/auth/session`，把页面上显示的整段内容完整复制出来（以 `{` 开头，里面有 `accessToken`）。只复制其中的 `token`、或者邮箱和密码，都不行（返回 `session_format`）。

```
curl -X POST {BASE}/redeem/verify -H "Authorization: Bearer $KEY" \
-H "Content-Type: application/json" \
-d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX","session":{"user":{"...},"accessToken":"...\"}}'
```

账号可以兑换：

```
{
  "ok": true,
  "data": {
    "eligible": true,
    "ticket": "tk_3f9a1c07b2d4e5a6b7c8d9e0f1a2b3c4",
    "expires_at": "2026-10-05T09:10:00Z",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "account": {
      "email": "bu***@example.com",
      "current_plan": "free",
      "current_plan_name": "Free",
      "has_active_plan": false
    }
  },
  "trace_id": "9F2A41C07B"
}
```

账号已经有生效中的套餐，**不能叠加**（HTTP 仍然是 200，但没有 `ticket`）：

```
{
  "ok": true,
  "data": {
    "eligible": false,
    "reason": "account_has_plan",
    "message": "这个账号已有生效中的套餐，不能叠加，请换一个账号。卡密未消耗",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "account": {
      "email": "bu***@example.com",
      "current_plan": "plus",
      "current_plan_name": "Plus",
      "has_active_plan": true
    }
  },
}
```



```
"trace_id": "9F2A41C07B"
}
```

字段	说明
<code>eligible</code>	<code>true</code> 才有 <code>ticket</code> 。 <code>false</code> 时请买家换一个账号（ <code>reason</code> 目前只有 <code>account_has_plan</code> ）。卡密没有消耗。
<code>ticket</code>	核对票据。 10 分钟内有效 （ <code>expires_at</code> ）， 只能用于一次兑换 。过期了重新核对。
<code>account.email</code>	打码后的账号邮箱，给买家确认「是不是这个账号」。
<code>account.current_plan</code>	账号现在的套餐： <code>free</code> 或上面 <code>plan</code> 表里的取值。
<code>account.has_active_plan</code>	账号现在有没有生效中的套餐。 <code>true</code> 时 <code>eligible</code> 一定是 <code>false</code> 。

可能的错误

code	什么时候	怎么办
<code>session_format</code>	Session 不是整页内容（缺 <code>accessToken</code> 等）	请买家按上面的方法重新复制
<code>session_invalid</code>	账号登录已失效	请买家重新登录后再复制一次
<code>cdk_invalid</code> / <code>cdk_used</code> / <code>cdk_not_usable</code>	卡密的问题	同 <code>/cdk/query</code>
<code>wait_before_retry</code>	这个账号刚失败过一次，要等一会儿（带 <code>retry_after</code> ）	等 <code>retry_after</code> 秒后重新核对
<code>account_not_eligible</code>	这个账号现在不能充值这个套餐	请买家换一个账号
<code>request_rejected</code>	请求没有通过	请买家核对卡密和账号后重试
<code>busy</code> / <code>service_unavailable</code> / <code>try_later</code>	系统繁忙或暂时不可用	稍后重新核对

POST /redeem

POST

/redeem

用核对票据确认兑换。**从这一步开始，卡密才会被占用。** 请求很快返回订单号，开通在后台进行。

请求字段	说明
ticket	/redeem/verify 返回的票据。
idempotency_key	幂等号：8-64 位，字母、数字和 . _ : - 。 每一次新的兑换尝试用一个新值（推荐用你系统里的订单号或 UUID）；同一次尝试的所有重试用同一个值。

```
curl -X POST {BASE}/redeem -H "Authorization: Bearer $KEY" \
-H "Content-Type: application/json" \
-d '{"ticket":"tk_3f9a1c07b2d4e5a6b7c8d9e0f1a2b3c4","idempotency_key":"order-20261005-000123"}'
```

```
{
  "ok": true,
  "data": {
    "order_no": "OD3F9A1C07B2D4E5",
    "status": "processing",
    "cdk_state": "held",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "buyer_email": "bu***@example.com",
    "message": "开通中，通常 1-3 分钟。请不要重复提交。卡密已锁定。",
    "created_at": "2026-10-05T09:02:11Z",
    "updated_at": "2026-10-05T09:02:11Z",
    "poll_after": 8
  },
  "trace_id": "9F2A41C07B"
}
```

返回的就是一张订单，字段含义见 [GET /orders/{order_no}](#)。

出了问题怎么重来（务必读完）

你遇到的情况	怎么做
请求超时、断网、没拿到响应，或收到 5xx	用 同一个 idempotency_key 、 同一张 ticket 原样再发一次，会返回同一张订单，不会重复兑换。也可以用 POST /orders/lookup 按卡密找回订单。 不要 换新的幂等号，不要重新核对后再提交。

返回「没送出去」类错误：`busy`、`too_many_inflight`、`service_unavailable`、`try_later`

这次提交没有发生，票据还能用。等 `retry_after` 秒（没有就等几秒）后，用**同一张票据、同一个幂等号**原样再发。

返回其他错误：`wait_before_retry`、`ticket_expired`、`session_invalid`、`account_has_plan`、`cdk_used`、`cdk_not_usable`、`request_rejected`

这张票据**已作废**。先处理问题（等一等 / 换账号 / 重新复制 Session），再从 `/redeem/verify` 重新来，并用**新的**幂等号。

`ticket_used`

这张票据已经提交过了，`error.details.order_no` 就是对应订单，去查它。换新的幂等号不会得到新订单。

`idempotency_key_reused`

这个幂等号已经和另一张票据一起用过。换一个**新的**。

GET /orders/{order_no}

GET /orders/{order_no}

查一张订单现在的状态。`order_no` 是 `/redeem` 返回的订单号。只能查你这把密钥提交的订单，别人的订单一律当作不存在（404）。

```
curl {BASE}/orders/OD3F9A1C07B2D4E5 -H "Authorization: Bearer $KEY"
```

```
{
  "ok": true,
  "data": {
    "order_no": "OD3F9A1C07B2D4E5",
    "status": "failed",
    "cdk_state": "available",
    "plan": "pro-5x",
    "plan_name": "Pro 5x",
    "buyer_email": "bu***@example.com",
    "message": "这次付款被风控拦截了，请过两分钟之后再尝试。卡密未消耗。",
    "created_at": "2026-10-05T09:02:11Z",
    "updated_at": "2026-10-05T09:03:42Z",
    "finished_at": "2026-10-05T09:03:42Z",
    "failure": { "code": "payment_blocked", "action": "retry_later" }
  },
  "trace_id": "9F2A41C07B"
}
```

字段	说明
<code>order_no</code>	订单号。
<code>status</code>	订单状态： <code>processing</code> / <code>succeeded</code> / <code>failed</code> / <code>under_review</code> ，见 状态与失败处理 。
<code>cdk_state</code>	卡密现在的状态： <code>available</code> / <code>held</code> / <code>used</code> 。
<code>plan</code> / <code>plan_name</code>	套餐，取值见 POST /cdk/query 。
<code>buyer_email</code>	打码后的买家账号邮箱。
<code>message</code>	这一单现在的情况，给买家看的中文句子（末尾说明卡密有没有消耗）。
<code>created_at</code> / <code>updated_at</code> / <code>finished_at</code>	提交时间 / 最近更新 / 结束时间（只有 <code>succeeded</code> 、 <code>failed</code> 才有 <code>finished_at</code> ）。
<code>failure</code>	只在 <code>failed</code> 时出现： <code>code</code> 是失败原因， <code>action</code> 是建议界面怎么引导买家。见下文。
<code>poll_after</code>	只在没出结果时出现：建议多少秒后再来查。 请不要比它查得更勤 ：同一张订单 6 秒内的重复查询只会拿到同样的结果。

POST /orders/lookup

POST

/orders/lookup

按卡密找回订单：订单号弄丢了、页面刷新了、或者提交时没拿到响应的时候用。返回**你这把密钥**对这张卡密提交的最近一张订单（格式同 `GET /orders/{order_no}`）；没有就是 `404 no_such_order`。别的代理提交的、或买家在我们网站上兑换的，查不到。

请求字段	说明
<code>cdk</code>	卡密。卡密放在请求体里、不放网址里，免得出现在各种访问日志中。

```
curl -X POST {BASE}/orders/lookup -H "Authorization: Bearer $KEY" \
-H "Content-Type: application/json" \
-d '{"cdk":"XXXXXXXXXXXX-XXXXXXXXXXXX-XXXXXXXXXXXX"}'
```

5. 状态与失败处理

订单状态 `status`

status	含义	你该怎么做
<code>processing</code>	开通中（通常 1–3 分钟）	按 <code>poll_after</code> 继续查； 不要重复提交 。卡密已锁定。
<code>succeeded</code>	已开通（终态）	告诉买家去 ChatGPT 里确认。卡密已消耗。
<code>failed</code>	失败（终态）	看 <code>failure</code> 和 <code>cdk_state</code> ： <code>available</code> 可以重新来；其他情况不要重新提交，联系我们。
<code>under_review</code>	需要人工核对（通常 2 小时内有结果）	不要重复提交；继续按 <code>poll_after</code> （60 秒）查，它之后会变成 <code>succeeded</code> 或 <code>failed</code> ；着急就联系我们并提供订单号。卡密已锁定。

卡密状态 `cdk_state`

cdk_state	含义
<code>available</code>	还没消耗：卡密还在，可以重新核对、重新提交。
<code>held</code>	已锁定：正在处理或等人工核对， 不能重复提交 。
<code>used</code>	已使用：已经兑换成功。

失败订单的 `cdk_state` 以最近一次查询为准：失败后卡密恢复成 `available` 可能要等几秒。

失败原因 `failure.code` 与建议动作 `failure.action`

failure.code	action	含义
<code>session_invalid</code>	<code>recopy_session</code>	账号登录已失效：请买家重新复制 Session，再重新核对。
<code>account_has_plan</code>	<code>change_account</code>	账号已有生效中的套餐：请买家换一个账号。
<code>account_restricted</code>	<code>change_account</code>	账号暂时无法充值：过两分钟再试或换账号。

payment_blocked	retry_later	付款被风控拦截：过两分钟再试。
payment_failed	retry_later	付款没有成功：过两分钟再试。
try_later	retry_later	通道暂时繁忙：过两分钟再试。
cancelled	retry_later	这次充值已取消。
not_accepted	retry_later	这次提交没有被受理。
recharge_failed	retry_later	其他原因没有完成。

失败之后怎么重来

订单 `failed` 且 `cdk_state` 是 `available`：卡密还在。按 `action` 引导买家（等两分钟 / 换账号 / 重新复制 Session），然后从 `/redeem/verify` 重新来，使用**新的**幂等号。部分失败（比如付款被风控拦截）之后，同一个账号需要等约 2 分钟才能再次提交，否则会收到 `wait_before_retry`（带 `retry_after`）。

`cdk_state` 不是 `available` 时，不要重新提交，联系我们。

6. 错误码总表

code	HTTP	含义	怎么办
invalid_key	401	密钥无效或已停用	检查密钥；已停用的联系我们
ip_not_allowed	403	来源 IP 不在这把密钥登记的名单里	用 <code>/ping</code> 看我们看到的 IP，联系我们登记
service_disabled	503	接口暂时关闭	稍后重试
rate_limited	429	请求过于频繁	按 <code>Retry-After</code> 等
too_many_failures	429	来自这个 IP 的错误请求过多，被临时封锁	等 <code>Retry-After</code> ，检查密钥和来源 IP
busy	429	系统繁忙	按 <code>Retry-After</code> 稍后重试
too_many_inflight	429	同时处理的订单已达上限	等已有订单出结果后再提交
bad_request	400	请求格式不对	检查请求体

<code>not_found</code>	404	接口不存在	检查路径和方法
<code>cdk_invalid</code>	422	卡密无效	请买家核对卡密
<code>cdk_used</code>	409	卡密已兑换或已使用	看 <code>details.order_no</code>
<code>cdk_not_usable</code>	409	卡密暂时不能用（停用、冻结或正在处理）	联系我们
<code>session_format</code>	400	Session 不是整页内容	请买家重新复制整页 Session
<code>session_invalid</code>	422	账号登录已失效	请买家重新复制 Session
<code>account_not_eligible</code>	409	账号现在不能充值这个套餐	请买家换账号
<code>account_has_plan</code>	409	账号已有生效中的套餐	请买家换账号
<code>wait_before_retry</code>	409	这个账号要等一会儿再试	等 <code>retry_after</code> 秒后重新核对
<code>no_such_ticket</code>	404	没有这张核对票据	重新核对账号
<code>ticket_expired</code>	409	核对已过期或已作废	重新核对账号
<code>ticket_used</code>	409	这张票据已经提交过	看 <code>details.order_no</code>
<code>idempotency_key_reused</code>	409	幂等号已用于另一张票据	换一个新的幂等号
<code>no_such_order</code>	404	没有这张订单	检查订单号；只能查本密钥提交的订单
<code>request_rejected</code>	422	请求没有通过	请买家核对卡密和账号后重试
<code>try_later</code>	503	请过两分钟之后再尝试	稍后重试
<code>service_unavailable</code>	503	服务暂时不可用	稍后重试
<code>internal_error</code>	500	服务内部错误	稍后重试；持续出现请带 <code>trace_id</code> 联系我们

来源 IP 不在登记的名单里时，回包里会写本站看到的你的来源 IP，照它登记就行：

```
{
  "ok": false,
```

```

"error": {
  "code": "ip_not_allowed",
  "message": "当前请求来源的 IP 不在这把密钥允许的名单里",
  "retryable": false,
  "details": { "ip": "203.0.113.99" }
},
"trace_id": "9F2A41C07B"
}

```

7. 示例代码

下面三份示例做的是同一件事：查卡密 → 核对账号 → 确认兑换（按 [出了问题怎么重来](#) 处理超时和「这次没处理」的错误）→ 按 `poll_after` 查到出结果。都只用语言自带的库，设好 `AGENT_API_KEY` 环境变量就能运行；发布前我们对着真实接口逐个跑通过。

接进你自己的系统时

- `idempotency_key` 要在调用 `/redeem` 之前和你自己的订单一起存进数据库；进程重启以后的重试也用它。
- 示例用 `sleep` 轮询是为了好读。在你的系统里通常是后台任务按 `poll_after` 定时去查，买家页面只显示当前的 `message`。
- 买家的 `Session` 只在这一次请求里用，不要保存、不要写日志。

▼ Python 3.8+ (`redeem.py`)

"""兑换接口 v1 示例 (Python 3.8+, 只用标准库)。

用法:

```

export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
python redeem.py <卡密> <买家 Session 文件>

```

把这份代码接进你的系统时：`idempotency_key` 要在调用 `/redeem` 之前和你自己的订单一起存进数据库，重试时原样复用；买家的 `Session` 只在这一次请求里用，不要保存、不要写日志。

"""

```

import http.client
import json
import os
import secrets
import sys
import time
import urllib.error
import urllib.request

```

```

BASE = os.environ.get("AGENT_API_BASE", "https://recharge.chongplus.store/api/open/v1")
KEY = os.environ["AGENT_API_KEY"]
# 换成你自己系统的名字。一定要带：urllib 的默认值 (Python-urllib/3.x) 会被网关直接拒绝 (HTTP 403, 回包是 error
code: 1010)。
USER_AGENT = "my-shop/1.0"

```



```
# 这几种错误说明「这次没处理」：等 retry_after 秒后，用同一张 ticket、同一个 idempotency_key 原样再发。
RESEND_SAME = {"busy", "rate_limited", "too_many_inflight", "service_unavailable",
               "service_disabled", "try_later", "internal_error"}
UNFINISHED = {"processing", "under_review"}
```

```
class ApiError(Exception):
    """接口明确返回了错误 (ok=false) 。”

    def __init__(self, status, error, trace_id):
        super().__init__(f"{error.get('code')}: {error.get('message')} (trace_id={trace_id})")
        self.status = status
        self.code = error.get("code", "")
        self.message = error.get("message", "")
        self.retry_after = error.get("retry_after") or 0
        self.details = error.get("details") or {}
```

```
class NetworkError(Exception):
    """没拿到响应 (超时、断网、网关错误页)：结果未知，只能原样重发或按卡密找回。"""
```

```
def call(method, path, body=None):
    data = None if body is None else json.dumps(body, ensure_ascii=False).encode("utf-8")
    req = urllib.request.Request(BASE + path, data=data, method=method)
    req.add_header("Authorization", "Bearer " + KEY)
    req.add_header("User-Agent", USER_AGENT)
    if data is not None:
        req.add_header("Content-Type", "application/json")
    try:
        with urllib.request.urlopen(req, timeout=90) as resp: # 核对账号一步要 10 秒左右
            status, raw = resp.status, resp.read()
    except urllib.error.HTTPError as e: # 4xx / 5xx 也带着 JSON
        status, raw = e.code, e.read()
    except (OSError, http.client.HTTPException) as e: # 超时、断网、连接被断开
        raise NetworkError(str(e)) from e
    try:
        payload = json.loads(raw)
    except ValueError as e:
        raise NetworkError(f"HTTP {status}, 回包不是 JSON") from e
    if payload.get("ok"):
        return payload["data"]
    raise ApiError(status, payload.get("error") or {}, payload.get("trace_id"))
```

```
def submit(ticket, idem, cdk):
    """确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。"""
    for _ in range(6):
        try:
            return call("POST", "/redeem", {"ticket": ticket, "idempotency_key": idem})
        except NetworkError:
            time.sleep(3)
        except ApiError as e:
            if e.code == "ticket_used" and e.details.get("order_no"):
```

```

        return call("GET", "/orders/" + e.details["order_no"]) # 这张 ticket 已经提交过: 查那张单
    if e.code not in RESEND_SAME:
        raise # 其他错误: 这张 ticket 作废, 处理完问题后从核对账号重新来, 并换新的幂等号
    time.sleep(e.retry_after or 3)
# 一直没拿到明确结果: 按卡密找回 (可能已经受理了), 绝不换新的幂等号重新提交。
return call("POST", "/orders/lookup", {"cdk": cdk})

def wait_until_done(order, deadline_seconds=2 * 3600):
    deadline = time.monotonic() + deadline_seconds
    while order["status"] in UNFINISHED:
        if time.monotonic() > deadline:
            break # 等太久了: 提示买家联系客服并给出 order_no, 不要重新提交
        time.sleep(order.get("poll_after") or 8)
    try:
        order = call("GET", "/orders/" + order["order_no"])
    except NetworkError:
        time.sleep(10)
    except ApiError as e:
        if e.code not in RESEND_SAME:
            raise
        time.sleep(e.retry_after or 10)
    return order

def redeem(cdk, session):
    info = call("POST", "/cdk/query", {"cdk": cdk}) # 可选: 先告诉买家是什么套餐
    print("套餐: ", info["plan_name"], file=sys.stderr)

    check = call("POST", "/redeem/verify", {"cdk": cdk, "session": session})
    if not check["eligible"]:
        return {"ok": False, "message": check["message"]} # 账号已有套餐: 请买家换一个账号

    idem = "ord-" + secrets.token_hex(12) # 实际使用时: 和你的订单一起先存库, 重试时原样复用
    order = wait_until_done(submit(check["ticket"], idem, cdk))
    return {
        "ok": order["status"] == "succeeded",
        "status": order["status"],
        "message": order["message"],
        "order_no": order["order_no"],
        "cdk_state": order["cdk_state"],
        "failure": order.get("failure"),
    }

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit("用法: python redeem.py <卡密> <买家 Session 文件>")
    with open(sys.argv[2], encoding="utf-8") as f:
        buyer_session = f.read()
    try:
        result = redeem(sys.argv[1], buyer_session)
    except ApiError as e:

```

```
result = {"ok": False, "code": e.code, "message": e.message, "details": e.details}
print(json.dumps(result, ensure_ascii=False, indent=2))
```

▼ Node.js 18+ (redeem.mjs)

```
// 兑换接口 v1 示例 (Node.js 18+, 不用装任何依赖) 。
//
// 用法:
//   export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
//   node redeem.mjs <卡密> <买家 Session 文件>
//
// 把这份代码接进你的系统时: idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库,
// 重试时原样复用; 买家的 Session 只在这一次请求里用, 不要保存、不要写日志。

import { randomBytes } from 'node:crypto'
import { readFileSync } from 'node:fs'
import { setTimeout as sleep } from 'node:timers/promises'

const BASE = process.env.AGENT_API_BASE || 'https://recharge.chongplus.store/api/open/v1'
const KEY = process.env.AGENT_API_KEY
// 换成你自己系统的名字。别用语言自带的默认值: 有的会被网关直接拒绝 (HTTP 403, 回包是 error code: 1010) 。
const USER_AGENT = 'my-shop/1.0'

// 这几种错误说明「这次没处理」: 等 retry_after 秒后, 用同一张 ticket、同一个 idempotency_key 原样再发。
const RESEND_SAME = new Set([
  'busy',
  'rate_limited',
  'too_many_inflight',
  'service_unavailable',
  'service_disabled',
  'try_later',
  'internal_error',
])
const UNFINISHED = new Set(['processing', 'under_review'])

/** 接口明确返回了错误 (ok=false) 。 */
class ApiError extends Error {
  constructor(status, error, traceId) {
    super(`${error.code}: ${error.message} (trace_id=${traceId})`)
    this.status = status
    this.code = error.code || ''
    this.userMessage = error.message || '' // 给买家看的那句话
    this.retryAfter = error.retry_after || 0
    this.details = error.details || {}
  }
}

/** 没拿到响应 (超时、断网、网关错误页): 结果未知, 只能原样重发或按卡密找回。 */
class NetworkError extends Error {}

async function call(method, path, body) {
  let res
  try {
```

```

    res = await fetch(BASE + path, {
      method,
      headers: {
        Authorization: `Bearer ${KEY}`,
        'User-Agent': USER_AGENT,
        ...(body === undefined ? {} : { 'Content-Type': 'application/json' }),
      },
      body: body === undefined ? undefined : JSON.stringify(body),
      signal: AbortSignal.timeout(90_000), // 核对账号一步要 10 秒左右
    })
  } catch (e) {
    throw new NetworkError(String(e))
  }
  let payload
  try {
    payload = await res.json()
  } catch {
    throw new NetworkError(`HTTP ${res.status}, 回包不是 JSON`)
  }
  if (payload.ok) return payload.data
  throw new ApiError(res.status, payload.error || {}, payload.trace_id)
}

/** 确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。 */
async function submit(ticket, idem, cdk) {
  for (let i = 0; i < 6; i++) {
    try {
      return await call('POST', '/redeem', { ticket, idempotency_key: idem })
    } catch (e) {
      if (e instanceof NetworkError) {
        await sleep(3000)
        continue
      }
      if (e.code === 'ticket_used' && e.details.order_no) {
        return call('GET', `/orders/${e.details.order_no}`) // 这张 ticket 已经提交过：查那张单
      }
      if (!RESEND_SAME.has(e.code)) throw e // 其他错误：这张 ticket 作废，处理完问题后从核对账号重新来，并换新的幂等号
      await sleep((e.retryAfter || 3) * 1000)
    }
  }
  // 一直没拿到明确结果：按卡密找回（可能已经受理了），绝不换新的幂等号重新提交。
  return call('POST', '/orders/lookup', { cdk })
}

async function waitUntilDone(order, deadlineMs = 2 * 3600_000) {
  const deadline = Date.now() + deadlineMs
  while (UNFINISHED.has(order.status)) {
    if (Date.now() > deadline) break // 等太久了：提示买家联系客服并给出 order_no，不要重新提交
    await sleep((order.poll_after || 8) * 1000)
    try {
      order = await call('GET', `/orders/${order.order_no}`)
    } catch (e) {
      if (!(e instanceof NetworkError) && !RESEND_SAME.has(e.code)) throw e
      await sleep((e.retryAfter || 10) * 1000)
    }
  }
}

```

```

    }
  }
  return order
}

export async function redeem(cdk, session) {
  const info = await call('POST', '/cdk/query', { cdk }) // 可选: 先告诉买家是什么套餐
  console.error('套餐: ', info.plan_name)

  const check = await call('POST', '/redeem/verify', { cdk, session })
  if (!check.eligible) return { ok: false, message: check.message } // 账号已有套餐: 请买家换一个账号

  const idem = `ord-${randomBytes(12).toString('hex')}` // 实际使用时: 和你的订单一起先存库, 重试时原样复用
  const order = await waitUntilDone(await submit(check.ticket, idem, cdk))
  return {
    ok: order.status === 'succeeded',
    status: order.status,
    message: order.message,
    order_no: order.order_no,
    cdk_state: order.cdk_state,
    failure: order.failure,
  }
}

if (process.argv.length === 4) {
  const [cdk, sessionFile] = process.argv.slice(2)
  try {
    console.log(JSON.stringify(await redeem(cdk, readFileSync(sessionFile, 'utf8')), null, 2))
  } catch (e) {
    if (!(e instanceof ApiError)) throw e
    console.log(JSON.stringify({ ok: false, code: e.code, message: e.userMessage, details: e.details }, null, 2))
  }
} else {
  console.error('用法: node redeem.mjs <卡密> <买家 Session 文件>')
  process.exitCode = 1
}

```

▼ Go 1.21+ (redeem.go)

```

// 兑换接口 v1 示例 (Go 1.21+, 只用标准库)。
//
// 用法:
//
//     export AGENT_API_KEY=cpa_...          # 密钥只放在服务器的环境变量里
//     go run redeem.go <卡密> <买家 Session 文件>
//
// 把这份代码接进你的系统时: idempotency_key 要在调用 /redeem 之前和你自己的订单一起存进数据库,
// 重试时原样复用; 买家的 Session 只在这一次请求里用, 不要保存、不要写日志。
package main

import (
    "bytes"

```

```

        "crypto/rand"
        "encoding/hex"
        "encoding/json"
        "errors"
        "fmt"
        "net/http"
        "os"
        "time"
    )

    var (
        base = envOr("AGENT_API_BASE", "https://recharge.chongplus.store/api/open/v1")
        key   = os.Getenv("AGENT_API_KEY")
        client = &http.Client{Timeout: 90 * time.Second} // 核对账号一步要 10 秒左右
    )

    // 换成你自己系统的名字。别用语言自带的默认值：有的会被网关直接拒绝 (HTTP 403, 回包是 error code: 1010) 。
    const userAgent = "my-shop/1.0"

    // 这几种错误说明「这次没处理」：等 retry_after 秒后, 用同一张 ticket、同一个 idempotency_key 原样再发。
    var resendSame = map[string]bool{
        "busy": true, "rate_limited": true, "too_many_inflight": true, "service_unavailable": true,
        "service_disabled": true, "try_later": true, "internal_error": true,
    }

    // APIError 接口明确返回了错误 (ok=false) 。
    type APIError struct {
        Status      int           `json:"- "`
        Code        string        `json:"code"`
        Message     string        `json:"message"`
        RetryAfter  int           `json:"retry_after"`
        Details     map[string]string `json:"details"`
        TraceID     string        `json:"- "`
    }

    func (e *APIError) Error() string {
        return fmt.Sprintf("%s: %s (trace_id=%s)", e.Code, e.Message, e.TraceID)
    }

    // ErrNetwork 没拿到响应 (超时、断网、网关错误页) : 结果未知, 只能原样重发或按卡密找回。
    var ErrNetwork = errors.New("network error")

    // Order 一张订单 (只列了这里用到的字段) 。
    type Order struct {
        OrderNo  string `json:"order_no"`
        Status   string `json:"status"`
        CDKState string `json:"cdk_state"`
        Message  string `json:"message"`
        PollAfter int    `json:"poll_after"`
        Failure  *struct {
            Code   string `json:"code"`
            Action string `json:"action"`
        } `json:"failure,omitempty"`
    }

```

```

func call(method, path string, body, out any) error {
    var reader *bytes.Reader
    if body != nil {
        raw, err := json.Marshal(body)
        if err != nil {
            return err
        }
        reader = bytes.NewReader(raw)
    } else {
        reader = bytes.NewReader(nil)
    }
    req, err := http.NewRequest(method, base+path, reader)
    if err != nil {
        return err
    }
    req.Header.Set("Authorization", "Bearer "+key)
    req.Header.Set("User-Agent", userAgent)
    if body != nil {
        req.Header.Set("Content-Type", "application/json")
    }
    resp, err := client.Do(req)
    if err != nil {
        return fmt.Errorf("%w: %v", ErrNetwork, err)
    }
    defer resp.Body.Close()
    var payload struct {
        OK      bool          `json:"ok"`
        Data    json.RawMessage `json:"data"`
        Error   *APIError     `json:"error"`
        TraceID string        `json:"trace_id"`
    }
    if err := json.NewDecoder(resp.Body).Decode(&payload); err != nil {
        return fmt.Errorf("%w: HTTP %d, 回包不是 JSON", ErrNetwork, resp.StatusCode)
    }
    if !payload.OK {
        e := payload.Error
        if e == nil {
            e = &APIError{Code: "internal_error"}
        }
        e.Status, e.TraceID = resp.StatusCode, payload.TraceID
        return e
    }
    if out == nil {
        return nil
    }
    return json.Unmarshal(payload.Data, out)
}

```

// submit 确认兑换。网络出错、或者是「这次没处理」的错误，都用同一张 ticket、同一个幂等号原样再发。

```

func submit(ticket, idem, cdk string) (*Order, error) {
    for i := 0; i < 6; i++ {
        var order Order
        err := call(http.MethodPost, "/redeem", map[string]string{"ticket": ticket,
            "idempotency_key": idem}, &order)
        if err == nil {

```

```

        return &order, nil
    }
    var apiErr *APIError
    switch {
    case errors.Is(err, ErrNetwork):
        time.Sleep(3 * time.Second)
    case errors.As(err, &apiErr) && apiErr.Code == "ticket_used" && apiErr.Details["order_no"] !=
"":
        // 这张 ticket 已经提交过: 查那张单
        err = call(http.MethodGet, "/orders/"+apiErr.Details["order_no"], nil, &order)
        return &order, err
    case errors.As(err, &apiErr) && resendSame[apiErr.Code]:
        time.Sleep(time.Duration(max(apiErr.RetryAfter, 3)) * time.Second)
    default:
        return nil, err // 其他错误: 这张 ticket 作废, 处理完问题后从核对账号重新来, 并换新的幂等号
    }
}
// 一直没拿到明确结果: 按卡密找回 (可能已经受理了), 绝不换新的幂等号重新提交。
var order Order
err := call(http.MethodPost, "/orders/lookup", map[string]string{"cdk": cdk}, &order)
return &order, err
}

func waitUntilDone(order *Order, deadline time.Duration) (*Order, error) {
    stop := time.Now().Add(deadline)
    for order.Status == "processing" || order.Status == "under_review" {
        if time.Now().After(stop) {
            break // 等太久了: 提示买家联系客服并给出 order_no, 不要重新提交
        }
        time.Sleep(time.Duration(max(order.PollAfter, 1)) * time.Second)
        var next Order
        err := call(http.MethodGet, "/orders/"+order.OrderNo, nil, &next)
        var apiErr *APIError
        switch {
        case err == nil:
            order = &next
        case errors.Is(err, ErrNetwork):
            time.Sleep(10 * time.Second)
        case errors.As(err, &apiErr) && resendSame[apiErr.Code]:
            time.Sleep(time.Duration(max(apiErr.RetryAfter, 10)) * time.Second)
        default:
            return order, err
        }
    }
    return order, nil
}

func redeem(cdk, session string) (map[string]any, error) {
    var info struct {
        PlanName string `json:"plan_name"`
    }
    if err := call(http.MethodPost, "/cdk/query", map[string]string{"cdk": cdk}, &info); err != nil {
        return nil, err
    }
    fmt.Fprintln(os.Stderr, "套餐: ", info.PlanName) // 可选: 先告诉买家是什么套餐
}

```



```

var check struct {
    Eligible bool `json:"eligible"`
    Ticket   string `json:"ticket"`
    Message  string `json:"message"`
}
if err := call(http.MethodPost, "/redeem/verify", map[string]string{"cdk": cdk, "session": session},
&check); err != nil {
    return nil, err
}
if !check.Eligible {
    return map[string]any{"ok": false, "message": check.Message}, nil // 账号已有套餐: 请买家换一个
    账号
}

buf := make([]byte, 12)
if _, err := rand.Read(buf); err != nil {
    return nil, err
}
idem := "ord-" + hex.EncodeToString(buf) // 实际使用时: 和你的订单一起先存库, 重试时原样复用
order, err := submit(check.Ticket, idem, cdk)
if err != nil {
    return nil, err
}
if order, err = waitUntilDone(order, 2*time.Hour); err != nil {
    return nil, err
}
return map[string]any{
    "ok": order.Status == "succeeded", "status": order.Status, "message": order.Message,
    "order_no": order.OrderNo, "cdk_state": order.CDKState, "failure": order.Failure,
}, nil
}

func envOr(name, fallback string) string {
    if v := os.Getenv(name); v != "" {
        return v
    }
    return fallback
}

func main() {
    if len(os.Args) != 3 {
        fmt.Fprintln(os.Stderr, "用法: go run redeem.go <卡密> <买家 Session 文件>")
        os.Exit(1)
    }
    session, err := os.ReadFile(os.Args[2])
    if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    result, err := redeem(os.Args[1], string(session))
    var apiErr *APIError
    if errors.As(err, &apiErr) {
        result = map[string]any{"ok": false, "code": apiErr.Code, "message": apiErr.Message,
"details": apiErr.Details}
    }
}

```

```
    } else if err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    out, _ := json.MarshalIndent(result, "", " ")
    fmt.Println(string(out))
}
```

8. 上线前自检

- 密钥只在服务器的环境变量里，没有出现在前端、日志、聊天记录里。
- 用 `GET /ping` 确认「我们看到的来源 IP」就是你登记的出口 IP。
- 所有请求的超时 ≥ 60 秒；核对账号一步不要因为慢就当成失败。
- 请求带的是你自己系统的 `User-Agent`，不是语言自带的默认值。
- 一次兑换尝试只生成一个幂等号，所有重试复用它；超时的时候不重新核对、不换幂等号。
- 轮询间隔不短于 `poll_after`；设一个总的等待上限（比如 2 小时），超过了提示买家联系客服并给出 `order_no`。
- 买家的 Session 不落库、不进日志；你自己的日志里只记 `order_no`、`trace_id`。
- 买家界面直接显示 `message`；失败时按 `failure.action` 引导买家。
- 收到 429 / 503 会按 `Retry-After` 退避，而不是立刻重试。
- 联调步骤里的几个错误分支都走过一遍，界面上给买家的提示和本文档一致。

9. 常见问题

核对账号为什么要 10 秒？

要实时校验买家的账号，这是正常的耗时。请把超时设长，界面上给买家一个「正在核对账号」的提示。

买家点了两次「确认兑换」会怎样？

一张票据只能用一次：同一个幂等号的第二次请求返回同一张订单；换了幂等号的第二次请求返回 `ticket_used`。不会重复兑换。

订单一直是 processing 怎么办？

通常 1–3 分钟出结果。超过 10 分钟请继续等并按 `poll_after` 查；超过 2 小时联系我们，并提供 `order_no`。不要重新提交。

买家想换一个账号重新兑换同一张卡密？

只要卡密还是 `available`（核对没通过、或兑换失败后卡密恢复），就可以用新的账号重新 `/redeem/verify`。

收到 403，回包却不是 JSON？

回包是一行 `error code: 1010`：请求在网关就被拒绝了，原因是 `User-Agent`（见「通用约定」），和密钥、IP 无关，换成你自己系统的名字就好。密钥或 IP 的问题，回包都是 JSON（`invalid_key`、`ip_not_allowed`）。

联系我们时需要提供什么？

`order_no`（或响应里的 `trace_id`）、发生的大致时间、卡密的前 8 位和后 4 位、你收到的完整响应（请去掉密钥）。

代理对接文档 · 兑换接口 v1 · 2026-10-05 首版。随附 `openapi.json` 和示例代码（Python / Node.js / Go）。变更会写在这里，接口的兼容性变动我们会提前通知。